ORCA — Repository Guide

For review before Phase 0 starts · August 2026

What each folder is, why it exists, and how the whole thing builds and runs. One decision at the end needs settling before the build agent runs.

---

1 · One decision first: where the Gradle root sits

The repository currently nests the Gradle project one level down:

```
orca/                    ← git root
├── docs/
└── orca/                ← Gradle root: settings.gradle.kts lives here
```

That works, and it is legal. It has four consequences worth accepting deliberately rather than discovering:

Every build command needs a `cd`	`cd orca && ./gradlew build`
CI needs a working-directory setting	Not the default, and easy to forget
IDE import is from a subdirectory	Fine, but a step people get wrong on first clone
`.gitignore` and `.gitattributes` exist twice	They are currently duplicated at both levels – the root pair is now dead and should go

The alternative is git root = Gradle root, which is what most Java monorepos do, and what removes all four.

The question that actually decides it: does the frontend live in this repository?

- **Backend only** → flatten. The extra level buys nothing.
- **Frontend joins later** — two React applications, the kiosk display mode, both builders — → **keep the nesting**, and the layout becomes `orca/` beside `console/`. That is a good structure, and retrofitting it once seven services exist is disruptive.

This guide describes the folders themselves; both layouts hold, and only the path prefix changes.

⚠️ **Whichever is chosen, delete the duplicate `.gitignore` and `.gitattributes`.** Two of each in one repository is a source of confusion nobody enjoys diagnosing.

---

2 · The structure

What exists today

```
orca/                                   git root
├── .gitattributes
├── .gitignore
├── docs/
│   ├── ORCA_ARCHITECTURE.md            the specification
│   ├── ORCA_OPEN_QUESTIONS_REGISTER.md  what is deliberately unsettled
│   ├── ORCA_PHASE0_BUILD_BRIEF.md      what the agent builds
│   └── REPOSITORY_GUIDE.md             this document
└── orca/                               Gradle root — the Spring Initializr project
    ├── build.gradle.kts
    ├── settings.gradle.kts
    ├── gradlew  ·  gradlew.bat
    ├── gradle/
    │   └── wrapper/
    └── src/
        ├── main/java/com/lynxis/orca/OrcaApplication.java
        ├── main/resources/application.yaml
        └── test/java/com/lynxis/orca/
```

One bootable application, generated. Nothing else.

What Phase 0 produces

```
orca/
├── .gitattributes
├── .gitignore
├── docs/
└── orca/
    ├── build.gradle.kts                AGGREGATOR — no src/, no application
    ├── settings.gradle.kts             13 modules registered
    ├── gradle.properties
    ├── gradlew  ·  gradlew.bat
    ├── gradle/
    │   ├── libs.versions.toml          one version catalog for every module
    │   └── wrapper/
    │
    ├── platform/                       THE PRIMITIVES — no domain types
    │   ├── outbox/
    │   │   ├── build.gradle.kts
    │   │   └── src/main|test/java/com/lynxis/orca/platform/outbox/
    │   ├── lease/
    │   │   ├── build.gradle.kts
    │   │   └── src/main|test/java/com/lynxis/orca/platform/lease/
    │   ├── scope/
    │   │   ├── build.gradle.kts
    │   │   └── src/main|test/java/com/lynxis/orca/platform/scope/
    │   ├── idempotency/
    │   │   ├── build.gradle.kts
    │   │   └── src/main|test/java/com/lynxis/orca/platform/idempotency/
    │   └── web/
    │       ├── build.gradle.kts
    │       └── src/main|test/java/com/lynxis/orca/platform/web/
    │
    ├── contracts/                      API source of truth
    │   ├── _shared.yaml                envelope · error object · pagination
    │   ├── orca-core.yaml
    │   ├── orca-runtime.yaml
    │   ├── orca-edge.yaml
    │   ├── orca-portal.yaml
    │   ├── orca-sync.yaml
    │   ├── orca-fleet.yaml
    │   └── orca-media.yaml
    │
    ├── services/                       SIX BOOTABLE APPLICATIONS
    │   ├── orca-core/
    │   │   ├── build.gradle.kts
    │   │   └── src/
    │   │       ├── main/java/com/lynxis/orca/core/
    │   │       │   ├── CoreApplication.java
    │   │       │   ├── api/
    │   │       │   ├── domain/
    │   │       │   └── persistence/
    │   │       ├── main/resources/application.yaml
    │   │       └── test/java/com/lynxis/orca/core/
    │   ├── orca-runtime/
    │   │   ├── build.gradle.kts
    │   │   └── src/
    │   │       ├── main/java/com/lynxis/orca/runtime/
    │   │       │   ├── RuntimeApplication.java
    │   │       │   ├── execution/     api/ · domain/ · persistence/
    │   │       │   ├── workitem/      api/ · domain/ · persistence/
    │   │       │   ├── integration/   api/ · domain/ · persistence/
    │   │       │   ├── notify/        api/ · domain/ · persistence/
    │   │       │   └── readmodel/     api/ · domain/ · persistence/
    │   │       ├── main/resources/application.yaml
    │   │       └── test/java/com/lynxis/orca/runtime/
    │   ├── orca-edge/       EdgeApplication   + api/ domain/ persistence/
    │   ├── orca-portal/     PortalApplication + api/ domain/ persistence/
    │   ├── orca-sync/       SyncApplication   + api/ domain/ persistence/
    │   ├── orca-fleet/      FleetApplication  + api/ domain/ persistence/
    │   └── orca-media/      README.md only — inherited, not built
    │
    ├── migrations/                     ONE module, seven schema folders
    │   ├── build.gradle.kts
    │   └── src/main/resources/db/migration/
    │       ├── platform/               outbox · lease · idempotency tables
    │       ├── core/                   migrates FIRST — publishes the views
    │       ├── runtime/
    │       ├── edge/
```

```
│       ├── portal/
│       ├── sync/
│       └── fleet/
│
├── build-checks/                   TESTS ONLY — output is build failures
│   ├── build.gradle.kts
│   └── src/test/java/com/lynxis/orca/checks/
│       platform purity · module walls · scope seam
│       error envelope · retention class
│
├── deploy/
│   ├── docker-compose.yml          SQL Server 2022 + Keycloak 26
│   ├── .env.example                committed — working local values
│   └── keycloak/realm-export.json  1 realm, 7 clients, service accounts
│
└── .github/workflows/ci.yml        build → test → integrationTest → check
```

13 Gradle modules. Six bootable applications. Zero business logic.

Only `orca-runtime` is decomposed into modules — those five are named by the architecture and the module wall depends on them. Every other service stays flat: the service package with `api`, `domain` and `persistence` directly beneath it.

---

3 · The five top-level folders

`platform/` – the shared primitives

Five modules, one implementation each: **outbox** · **lease** · **scope** · **idempotency** · **web**.

These are not utilities. **Every correctness guarantee in the architecture is a property of one of them** — nothing lost on restart, nothing half-applied, exactly one visit per truck, no query escaping its scope. That is why they are built before any service exists: four developers each starting a service would produce four subtly different versions of all five in their first week, which is precisely the defect class this rebuild exists to remove.

The rule that keeps it healthy: `platform/` holds primitives, never domain. If a class there knows what a visit, a lane, a ticket or a driver is, it belongs in a service. This is enforced by a build check, not by review — a shared module that accumulates domain logic becomes the thing everything depends on and nobody can change.

`services/` – seven modules, six deployable applications

Each service module is **its own Spring Boot application**: its own `main`, its own configuration, its own image, deployed independently. They share a repository and a version catalog, nothing else.

MODULE	WHAT IT OWNS
`orca-core`	The world as configured: sites, lanes, users, devices, workflow and screen design
`orca-runtime`	The world as it happens: the process engine, visits, work items, connectors
`orca-edge`	Every hardware contract, the capture buffer, the command log
`orca-portal`	Carriers, drivers, tickets – the only internet-facing service
`orca-sync`	Replication between a site and a hosted tier
`orca-fleet`	Licence issuance and signing – cloud only, never at a customer site
`orca-media`	Video and intercom – **inherited, not built here.** A README placeholder

Only `orca-runtime` is decomposed into modules — `execution`, `workitem`, `integration`, `notify`, `readmodel`. Those five come from the architecture and the module wall depends on them. The other services stay flat until their modules are defined; inventing a decomposition now would be a guess written as structure.

Inside every module, the same three packages: `api` (controllers and DTOs) · `domain` (entities and domain services) · `persistence` (repositories). This is not a style preference — it is what makes the module wall expressible as one rule: *no*

module may reference another module's `persistence` package.

`contracts/` – the API source of truth

Eight OpenAPI documents: one per service, plus `_shared.yaml` holding the response envelope, the error object, the error codes and pagination.

The API layer is generated from these, not the other way round. The generator emits an **interface** and its DTOs; the controller is hand-written and implements that interface. Change the contract and the build breaks until the controller matches.

That is what makes contract-first real rather than aspirational — the API surface stops being documentation and becomes a compile-time constraint.

`_shared.yaml` matters more than it looks: without one shared definition, seven services each invent their own error schema and the single-envelope guarantee decays into a convention within a month.

`migrations/` – one module, seven schema folders

One database, seven schemas, each written by exactly one service — enforced by database credentials, not by convention.

It is one Gradle module rather than seven because the ordering is a constraint: `core` migrates first, since it publishes the read-only views the other services consume, and a dependent migration cannot run before the view it reads exists. Split per service and that ordering becomes a race between developers.

`build-checks/` – the enforcement

A module containing **only tests, no production code.** Its entire output is build failures.

CHECK	FAILS WHEN
Platform purity	A class in `platform/` references a domain type
Module walls	A module reads another module's `persistence` package
Scope seam	A query is constructed outside the seam
Error envelope	A controller returns a shape other than the envelope
Retention class	A traffic-growing table has no declared retention class

This folder is why the monorepo is the right choice. These checks have to see every service at once. In seven separate repositories they degrade into a code-review convention — and a convention is what the current system enforced tenancy with, across 816 hand-written conditions.

---

4 · The supporting folders

`deploy/` — `docker-compose.yml` (SQL Server and Keycloak), a committed `.env.example` with working local values, and the Keycloak realm export: one realm, one client per service, service accounts for service-to-service calls. A developer clones, copies one file, and runs.

`.github/workflows/` — build → unit tests → integration tests on Testcontainers against real SQL Server → all build checks → coverage. A pull request cannot merge with any of them failing.

`docs/` — the architecture document (the specification), the open-questions register (what is deliberately unsettled), the Phase 0 brief (what the agent builds), and this guide.

---

5 · How it builds

One Gradle multi-project. **The root is an aggregator and holds no code** — no `src/`, no application. It sets the Java 25 toolchain and owns `gradle/libs.versions.toml`, the single version catalog every module draws from, so no two modules can disagree about a dependency version.

The Spring Boot plugin is declared at the root with `apply false` and applied in each service module. That is what makes six bootable applications instead of one.

```
./gradlew build              everything
./gradlew test               unit tests
./gradlew integrationTest    Testcontainers, real SQL Server
./gradlew check              the five build checks
./gradlew bootRun -p services/orca-core
```

---

6 · How it runs

Locally: `docker compose up` gives SQL Server and Keycloak; each service runs from the IDE or `bootRun`. Every service validates tokens **by signature, locally** — no call-out to Keycloak per request, which is also what lets a site keep working when the wide-area link drops.

At a customer site: the images for that deployment profile run on one server, or on more than one. A profile says *which* services run; it does not say how many instances of each. The platform is designed for more than one instance in every profile.

There is no message broker. Services hand work to each other through the database — the outbox in `platform/`. That is one fewer server to provision, patch and monitor at every site, and it is why `platform/outbox` is the most load-bearing module in the repository.

---

7 · What Phase 0 delivers, and what it does not

Delivers: a repository that builds, a local stack that runs, seven schemas that migrate, five primitives with tests that prove their properties, five build checks that fail the build when violated, CI, and six services that boot and report health.

Does not deliver: any product behaviour. No visit can start, no gate can open, no screen exists. Nothing is demonstrable to a customer.

⚠️ **Worth saying out loud before it starts.** Two to three weeks with four developers and nothing to show is entirely reasonable if it was announced and uncomfortable if it was not. The first demonstrable thing — a plate read producing a visit and a confirmed barrier — comes at the end of Phase 1.

What you are buying is that the guarantees move from a document into the build. After Phase 0 a developer *cannot* write a query that escapes its scope or publish a fact without recording it, because the build stops them. Before Phase 0, all of it depends on everyone remembering.

---

8 · Questions worth raising in review

1. **Does the frontend live in this repository?** It decides §1, and it is cheaper to decide now than after seven services exist.
2. **Is `platform/` the right set of five?** They are the primitives the architecture's guarantees rest on. If your lead sees a sixth, better to know before they are built.
3. **Is one database with seven schemas acceptable to whoever will operate it?** It is a deliberate choice — it is what allows in-transaction reads across services and removes the broker. A database per service would forbid both.

4. **Who owns `platform/` after Phase 0?** It is shared code with no natural owner, and shared code with no owner is how it drifts.
